

QuakeWatch: Project Document

Oct 3, 2026 · @Karthik Mohan · Release v0.1.0

QuakeWatch is a compact Snowflake batch warehouse for comparing USGS earthquake records, and their revisions, around public locations. It is retrospective analysis, not an earthquake warning, risk score, or damage assessment; distance and magnitude alone do not estimate shaking or impact.

Status: built, released as v0.1.0, and verified. A verification run on 2 Oct 2026 re-ran the checks from a clean clone against the live warehouse. This document keeps the original 29 Sep 2026 plan, its estimates and its targets, and records next to each what was measured, met, missed, or still open. Measured results come from the repository's evidence log (github.com/kaarthikmohaan/quakewatch).

Problem and goal

Today, an analyst comparing seismic history around, say, three facilities must open the USGS Earthquake Catalog, pick a region or radius, download GeoJSON or CSV exports, and eyeball the results. If USGS later revises a magnitude or relocates an event, the snapshot is stale and the comparison cannot be reproduced. QuakeWatch loads source records in bounded batches, preserves revisions, and provides reproducible comparisons in Snowflake.

User: a facilities-planning or community-preparedness analyst comparing historical USGS event records around candidate sites. Seattle, San Francisco, and Anchorage are public demonstration locations only, using public city-center coordinates, a 250 km radius, and a five-year event-time window, all stored in example configuration.

Why it matters: for retrospective siting or preparedness reviews, the difference between a preliminary M4.3 and a final M4.7 matters. QuakeWatch turns an ad-hoc, manual, non-reproducible exercise into a repeatable, auditable data product. It remains historical analysis, not safety guidance; USGS maps and notifications remain strong alternatives.

Promise	What it means
Loads source records in bounded batches	USGS catalog data is fetched through the FDSN event service in finite time windows instead of an unbounded dump. The FDSN count method sizes each window below the 20,000-result cap, so each load is verifiable and resumable and does not overwhelm the source. The five-year history is 180 month-sized windows across the three sites.
Preserves revisions	USGS routinely updates an event's magnitude, depth, and coordinates after the first automatic solution (automatic → reviewed → revised). Each version is stored as a separate revision keyed by the USGS event ID, so the analyst can see what was first reported and what the final reviewed value is.

Promise	What it means
Reproducible comparisons in Snowflake	Versioned records and reviewed SQL mean the same query on the same sites against the same warehouse state gives identical results. The 2 Oct verification re-ran the site-by-year query and matched every recorded count: no more "I downloaded this last Tuesday and don't remember which magnitude threshold I used."

In scope	Out of scope
Manually invoked bounded batch extracts from USGS FDSN GeoJSON	Claims of a continuous stream, warning, prediction, risk score, shaking, or damage
Historical comparisons around Seattle, San Francisco, and Anchorage, within 250 km and over a five-year event-time window	Private coordinates in Git or public demo outputs
Revision-aware historical facts, curated dimensions, and analytical views	Guaranteed complete coverage without audited source ranges
Reproducible backfills, idempotent loads, quality checks, and run evidence	Production uptime or a production SLO
Snowflake batch warehouse with Python extraction and Snowpark transformations	Snowpipe Streaming, Kafka, Spark, or a second cloud warehouse

Definition of success

Acceptance targets from the plan, with their status on 2 Oct 2026:

Target	Status
Each ingestion batch records its requested time range, query parameters, count-before/count-after values, row counts, fetch time, outcome, and any incomplete window	Met: manifests and <code>BATCH_ATTEMPT</code> receipts for all 179 batches
Raw data preserves the full source record and request/batch metadata	Met: the full GeoJSON payload is stored as <code>VARIANT</code> with attempt and file metadata
Re-running an identical batch does not create duplicate logical event revisions or change aggregate counts	Met for tested cases: a same-attempt rerun merged zero revisions; live checks find zero duplicate keys

Target	Status
A revision or deletion to an old event outside the recent origin-time window is found by the update sweep, creates a historical revision or tombstone, and updates the current-event view	Partly met: proven on synthetic fixtures; the live catalog-wide sweep has not completed (issue #4)
A deleted event is retained as a tombstone; an older backfill cannot resurrect it	Met for fixtures: the tombstone hid the current event and a stale replay did not restore it
A failed transform can be retried from already loaded raw data without refetching the API	Met: two live drills retried from RAW with identical counts and no refetch
The <code>updatedafter</code> watermark advances only after every bounded query window is loaded and reconciled; incomplete sweeps remain retryable	Held: the sweep is incomplete, so the watermark has correctly not advanced. It lives in Snowflake and advances by compare-and-set
Fact and dimension grains, keys, and joins are documented and checked with SQL	Met: data dictionary and ERD; zero duplicate key groups
Requested/loaded/rejected rows, pending batches, coverage gaps, source age, and fetch-to-curated latency are reported separately	Met: health and reconciliation views; latency measured separately
A sample period and target are set before measuring; observed results and sample size are published, not targets as results	Met: the 24-hour p95 target was set first; the 35.9-hour result was published as a miss
Before expanding beyond the thin slice, one facilities-planning analyst completes a five-year site/magnitude comparison and reviews its revisions and coverage notes. The manual baseline and actual task time are recorded, any improvement target is set before comparing, and the product framing continues only if the user confirms the task is useful	Not done: no participant recruited yet (issue #9)

Scope and MVP

The MVP is one batch path that must work end to end before any optional demonstration is built.

MVP: USGS FDSN → Python batch runner and manifest → Snowflake internal stage → `COPY INTO` raw VARIANT → Snowpark validation and revision history → facts, dimensions, and current view → reconciliation and health views. No streaming, no Airflow, no Cortex in the core path.

Core MVP	After the MVP	Out of scope
Bounded FDSN batch extraction with overlap, window audit, and manifest	Clone isolation and Time Travel recovery drill in a sandbox	Safety guidance or alerting service
Internal stage + <code>COPY INTO</code> into raw VARIANT	Optional, evaluated on-demand Cortex brief	Snowpipe Streaming, Kafka, Spark, or Airflow
Snowpark validation, revision history, dimensions, and current view	GitHub Actions CI and repeatable SQL bootstrap	Full infrastructure-as-code platform
Reconciliation, rejects, coverage and latency views	Optional local metrics endpoint	Private coordinates or unmeasured production claims

Status: everything in the Core MVP column is built. From After the MVP, the clone and Time Travel drill, the evaluated Cortex brief, GitHub Actions CI, and a repeatable SQL bootstrap (`make bootstrap`) are done; the optional local metrics endpoint was not built.

Estimate: roughly 4–6 evenings for the MVP if the Snowflake account and role are ready; roughly 6–8 evenings in total if account setup, recovery, tests, and documentation are new. These were estimates, not measured build times. **Actual:** the core build and live evidence took four calendar days, 29 Sep to 2 Oct 2026; hours were not tracked, so no time-saved claim is made.

Requirements and constraints

Questions the warehouse answers

- How do event counts and magnitude distributions change over time within a configured radius of each example site?
- Which event records were revised or deleted, and when did the warehouse observe those changes?
- How much time passed between source update, batch fetch, and curated availability?
- Which requested time windows have not been loaded successfully?

Distance and magnitude are descriptive filters only. The project does not estimate shaking, risk, damage, or recommended action.

Constraints

Constraint	Value
Source	USGS FDSN event API: bounded results, a 20,000-result cap per query, and no documented fixed global quota, uptime SLA, or transactionally consistent catalog snapshot. Neither quota nor uptime is treated as guaranteed
Runtime	Python 3.12 for the local HTTPX/Connector batch runner; Snowpark runs only inside the Snowflake Python procedure. Snowflake lists Python 3.12 as generally available. Python 3.12 and snowflake-snowpark-python 1.55.0 were confirmed in <code>SNOWFLAKE.INFORMATION_SCHEMA.PACKAGES</code> on 29 Sep 2026, and the procedure runtime, package, and local Connector versions were pinned
Hardware	Snowflake's Snowpark setup guide documents a local Snowpark/PyOpenSSL issue on Apple M1. The design avoids local Snowpark imports; any future local Snowpark use is tested separately on the actual Mac architecture
Compute and cost	XS warehouse with auto-suspend and short sessions; no clustering or special partitioning without query-profile evidence
Privacy	Public city-center example coordinates only; private site coordinates and account secrets never enter Git or public demo outputs
Licensing	Most USGS-produced information is public domain. Credit the U.S. Geological Survey and link to official event records; check rights before reusing third-party graphics

Risks and known limits

The biggest risk is claiming completeness the source cannot prove, so every gap is reported rather than hidden.

Risk	Mitigation	What happened
Source changes during a sweep, or a query window exceeds the cap	Split windows, compare counts, repeat the overlap, and disclose unresolved coverage gaps; never claim a consistent source snapshot	Windows 12, 42, and 73 timed out even after splitting and are reported as gaps
Source publication delay	Show source age separately from local pipeline latency	—

Risk	Mitigation	What happened
Account, package, or region access unavailable	Verify early; report the blocker; do not claim a working Snowflake path	Cortex was blocked on the trial account until a payment method was added
Rekey or delete creates stale or duplicate state	Preserve aliases and status; test tombstone and stale replay	Tombstone and stale-replay fixtures passed
Transform failure after raw load	Retry from durable raw rows and the manifest	Demonstrated twice in isolated drills
Schema drift	Preserve VARIANT, version the parser, test and replay in a sandbox	Done for parser version 2 on 2 Oct 2026
Cortex unavailable or unsafe	Deterministic SQL fallback; no safety language	4 of 13 briefs were rejected and replaced by SQL facts
Private site coordinate	Public examples only; private values stay local and ignored	—
Small sample	State dates and row/batch counts; keep targets separate from results	—
USGS placeholder (stub) records	Reject as <code>source_stub_record</code> ; each is an alias of a full record, so nothing is lost	485 records with no time, magnitude, or status found and rejected

Source behavior and data contract

The initial history load and the update sweep are separate, because event-time and update-time filters answer different questions. [USGS documents separate event-time and update-time filters, the 20,000-result cap, and the count method.](#)

Source behavior	Design effect
API query returns a bounded result set	Each logical request has stable parameters; every invocation or retry gets its own attempt ID, query-window counts, row counts, and status
Events may be revised after origin time	Keep origin, source-updated, fetched, and curated clocks separately
Deleted records can be returned with status	Preserve the status and model a tombstone; do not infer deletion from absence

Source behavior	Design effect
Results can be late or out of order	Use source update time and deterministic tie-breakers, not event time alone
Preferred ID and associated IDs can change	Preserve source IDs and resolve aliases before canonical current-state reporting
Additive fields may appear	Keep complete source JSON in VARIANT; parser versions describe typed projections
Invalid row vs invalid whole response	Row-level rejects retain their reason; a malformed response fails the batch and preserves raw prior state

History load

- Fixed five-year origin-time windows around each configured site.
- The FDSN count method sizes each window; windows are split until each query is comfortably below the 20,000-result cap, then fetched in one response without offset paging.
- If a response reaches the cap or the before/after counts differ, split and retry. A window that cannot be split further or does not reconcile is left as an explicit coverage gap.

Update sweep

- Each sweep records a fixed `sweep_started_at` and queries `updatedafter = last_committed_watermark - overlap`, with `starttime` at the configured USGS catalog lower bound and `endtime` fixed at the cutoff.
- No spatial or magnitude filters, so a revised event that moves into or out of a site's radius is still observed.
- The committed watermark advances only to `sweep_started_at`, and only after every window reconciles; the overlap repeats on the next run.
- Changes made while a sweep runs are meant to be picked up by the next overlapping sweep. The endpoint has no snapshot boundary, so residual gaps remain possible and are reported.
- Site distances are recomputed in the warehouse. Updates are kept for events within the five-year horizon or previously relevant canonical IDs, including deletions. The catalog lower bound and watermark are recorded in the manifest.
- This is best-effort reconciliation, not proof of a gap-free source snapshot.

What happened in practice

Observation	Detail
Leaf request size	Each leaf request targets fewer than 10,000 features, well under the cap
Time budget	Each window has a 180-second budget shared by its retries and splits; reconciled child windows are saved as checkpoints, so a retry does not refetch them
History gaps	Windows 12, 42, and 73 still timed out after splitting to three-hour slices and are reported as gaps
First live update sweep	Reconciled 79 child windows and stopped on 9 Oct 2023, where both the count and event endpoints time out for one minute (02:21–02:22 UTC); tracked as issue #4

Architecture

Separate Python commands, wrapped by make targets, perform extraction (JSONL files and an attempt manifest), upload to a Snowflake internal stage, `COPY INTO`, load-count reconciliation, and the Snowpark procedure call through the Snowflake Python Connector.

```
USGS FDSN GeoJSON API
-> Python batch runner: origin-time load + update-time sweep + attempt manifest
-> JSONL files in Snowflake internal stage
-> COPY INTO RAW_EVENT_RECORDS (payload VARIANT)
-> reconcile COPY results, then invoke Snowpark procedure
-> Snowpark validation and revision resolution
-> FACT_EVENT_REVISION + BRIDGE_EVENT_SITE + EVENT_CURRENT view + dimensions
-> SQL analysis and health views; Cortex brief checked against SQL
```

The initial historical load covers the five-year event-time horizon around the example sites: 177 of 180 windows loaded, with three recorded source gaps. Later runs use the catalog-wide `updatedafter` sweep, which is built and tested but has not yet completed live.

There is no Snowpipe Streaming, Snowflake Stream, or claim of sub-minute streaming. A batch load is the core path; Cortex and recovery drills are optional follow-on demonstrations.

Orchestration: one explicit orchestrator. Python performs extraction, upload, `COPY INTO`, reconciliation, and the procedure call; a Snowflake Task does not notice local files or start the Python extractor. A later scheduler may invoke the whole runner, and Snowflake Tasks are optional for warehouse-only steps after loading. No Airflow for this small single-warehouse graph. Runs are still manual; scheduling with failure alerts is issue #6.

References: [Snowpark runtime guidance](#), [package catalog](#).

Data design

Only the tables needed by three demonstration queries are kept: (1) event counts and magnitude bands by year and site; (2) revision counts and source-update-to-fetch delay by site and magnitude type; (3) latest non-deleted records for the selected date window. Each grain is stated and uniqueness is enforced in SQL, because Snowflake standard-table key constraints are not enforced by themselves.

Tables and writers

Object	Grain / writer	Purpose
BATCH_ATTEMPT	One row per extract/load execution; Python runner	Stable logical batch ID, unique attempt ID, requested range, query parameters, query windows, times, load outcome, counts, and gaps
RAW_EVENT_RECORDS	One returned source feature per batch-attempt/query-window row; <code>COPY INTO</code>	Full payload VARIANT plus attempt ID, fetched time, file, and row metadata
STG_EVENT_REVISION	One validated source observation; Snowpark procedure	Typed projection, parser version, canonical/associated IDs, reject status
BATCH_PROCESS_ATTEMPT	One transformation invocation per loaded attempt; Snowpark procedure	Unique process-attempt ID, start/end, status, counts, and failure details; retrying processing does not refetch or rewrite the extract manifest
DIM_DATE	One calendar date; warehouse build	Role-playing event-date and source-update-date slicing
DIM_SITE	One configured public location; warehouse build	Public coordinates and descriptive radius; no private site values
DIM_MAGNITUDE_TYPE	One magnitude scale/type; warehouse build	Correctly labels magnitude semantics
DIM_EVENT_STATUS	One source status; warehouse build	Reviewed, automatic, deleted/tombstone, and other documented values
FACT_EVENT_REVISION	One distinct canonical event revision; warehouse build	Event/source clocks, magnitude, coordinates, place, depth, status, source IDs, payload hash

Object	Grain / writer	Purpose
BRIDGE_EVENT_SITE	One event revision × configured site; warehouse build	Epicentral distance and within-radius flag for each example location
FACT_BATCH_RUN	One batch attempt; warehouse build	Fetches, loads, processes, rejects counts, latency, outcome, and gaps
EVENT_CURRENT	One canonical event after alias and revision rules; view	Latest non-stale, non-tombstoned state
Reject and health views	One rejected row or one health snapshot; views	Visible quality issues, pending work, coverage, source age, and latency
UPDATE_WATERMARK	One row per named update sweep; update loader	Committed <code>updatedafter</code> watermark, advanced by compare-and-set only after the sweep's RAW receipt and per-window counts reconcile

Date, site, magnitude-type, and status dimensions are used only where the three queries use them. Revision history lives in a fact table rather than overwriting a single row. Distance uses a documented horizontal great-circle formula from longitude/latitude: it is epicentral distance, not hypocentral distance or shaking.

Live checks on 2 Oct 2026: 216,391 rows reconciled across 179 batches, zero duplicate revision or site-bridge keys, and 485 rejects, all USGS placeholder records labelled `source_stub_record`.

Revision, alias, deletion, and schema rules

- Retain USGS origin time, source updated time, fetched time, and curated time.
- Resolve the current revision by source updated time, then fetched time and a stable payload hash as tie-breakers.
- Preserve preferred and associated IDs; reconcile aliases so rekeys do not create duplicate current events.
- Preserve status. A deleted record creates a tombstone; a stale backfill must not resurrect it.
- Do not discard a valid record only because its origin time is old.
- VARIANT retains unknown fields; the parser version tracks the typed projection.
- Invalid rows become rejects with a reason and count. A malformed whole response fails the batch.
- On a contract change, bump the parser version, replay retained raw data in a sandbox, compare outputs and reject counts, then promote. Done for parser version 2 on 2 Oct 2026: replaying all 216,591 local rows changed only the 485 placeholder rejects, which were relabelled `source_stub_record` in one checked transaction.

Guarantees and reprocessing

Loads are idempotent: re-running a batch converges to the same facts and counts, and every attempt, including failures, stays on record.

Batch loading and processing

1. Create a deterministic `logical_batch_id` from the requested query window and parameters, and a unique `attempt_id` for every extract/load invocation, including retries. Append one `BATCH_ATTEMPT` row per attempt and never overwrite a failed attempt's record. Give every file and row a stable source key.
2. Extract bounded origin-time windows for history; for updates, use `updatedafter` from the last successful watermark minus overlap, with the catalog-wide lower bound and no spatial or magnitude filters. Persist the manifest and JSONL files before loading.
3. Upload files to an internal stage and `COPY INTO` raw records, retaining file names and load metadata for reconciliation.
4. Dedupe repeated source observations before `MERGE`. Logical revision identity is canonical event identity, source update time, and payload hash; duplicate unseen source keys are tested explicitly.
5. In one transaction, merge validated revision rows and append a successful `BATCH_PROCESS_ATTEMPT` record. On failure, roll back fact changes, catch the error outside that transaction, and append a failed processing-attempt record separately.
6. Build dimensions and facts from durable raw and staging data.
7. Reconcile requested query windows to raw rows and processed/rejected results. HTTP 200 alone never marks a batch complete, and a successful retry does not erase the earlier failure.

One writer per table: the Python runner and `COPY` own `RAW_EVENT_RECORDS` and `BATCH_ATTEMPT`; the Snowpark procedure owns staging, facts, dimensions, and `BATCH_PROCESS_ATTEMPT`; views own current, reject, and health outputs. DDL stays outside processing transactions. Pending batches are exposed and retried from durable raw rows. Snowflake Tasks are post-MVP only.

Verified by a same-attempt rerun that merged zero revisions, two failed-transform drills, and zero duplicate key groups in production.

Recovery evidence

1. Load a batch, fail a transform, and show the durable raw data and the failed/pending manifest.
2. Retry processing without refetching; show unchanged logical revision counts and no duplicate fact keys.
3. Replay an overlapping batch and show convergence.
4. In a sandbox, clone only the small fact table, mutate the clone, and confirm the source is unchanged.

5. For Time Travel, suspend the sandbox writer, capture the inner `MERGE` query ID, make a deliberate bad merge, and query or clone `BEFORE` that statement; show before/after counts and recovery from raw.

All five were demonstrated on 1–2 Oct 2026 in an isolated fixture database: failed-transform retry from RAW (twice), an overlapping replay that added no duplicate revision, clone isolation, and a statement-ID Time Travel read. The demo clones were dropped.

Standard Time Travel retention is one day; longer retention for permanent tables requires Enterprise or higher, and statement-ID access has a separate 14-day limit. Clones start zero-copy, but divergence and history can incur storage, so demo clones are dropped.

Cortex brief

Cortex is explicit and on demand. It may turn a bounded analytical result into a short factual summary: event count in a selected window, nearest event, magnitude, record age, and source status. It must not infer risk, shaking, damage, or action, and it stays only if a user test shows it is easier to scan than the deterministic SQL result.

Controls

- Cap rows and tokens; allow one bounded retry; cache by aggregate hash.
- Persist model, time, and input hash when a brief is saved.
- Deterministic SQL fallback on access errors, failures, or unsafe output.
- Evaluate about 10 saved aggregates, check every fact against SQL, and reject hazard or action language.
- Cortex usage history can lag by up to five minutes, and warehouse credits are separate; no cost is claimed before measuring.

Result (1 Oct 2026)

Measure	Result
Evaluated briefs	Ten aggregates saved; claude-haiku-4-5 produced nine briefs, and all nine passed automatic validation and human fact review
Rejected briefs	Four others (two before the evaluation, two in the 2 Oct demo) misstated a fact, such as measuring distance from the event ID instead of the site; the SQL facts were shown instead
Usage	The first eleven completed calls used 0.003948756 AI credits
Preference	One side-by-side comparison by the author preferred the brief; without a user test it stays optional

Quality, security and cost

Check	What it covers	Runs
Fixture tests	Timestamps, coordinates, missing IDs, unknown fields, late arrival, repeated/overlapping batches, an old-event update outside the recent window, an event moving into/out of a site radius, revision winner, alias rekey, tombstone/stale replay, malformed response, row rejects, duplicate unseen merge keys, over-limit window splitting, count mismatch/retry, unresolved-window gap reporting, and watermark not advancing on failure. 340 tests pass, covering 86% of the Python package	Every push
CI	Locked install, ruff lint and format checks, mypy, the synthetic fixture build, and the secret-free tests under coverage (minimum 80%); a real status badge is shown	Every push and pull request
Snowflake integration	Compiles every procedure and reviewed SQL statement against the live schema with EXPLAIN, confirms the procedure exists, and runs the read-only reconciliation and uniqueness checks. First runs passed with 35 of 35 statements. Reads credentials from repository secrets and never writes data	Manual GitHub workflow
Recorded live drills	<code>COPY</code> repeat behaviour, failure and retry, and the merge-plus-log transaction	Once, with evidence

Security

- Key-pair authentication kept outside Git; least privilege through a project role whose key is restricted to it.
- Credentials stay in each person's Snowflake config, documented by `snowflake-config.example.toml` instead of an `.env.example`.
- GitHub push protection, CodeQL, and Dependabot are on; new commits are signed.
- Never commit a private site coordinate or account secret. A full-history scan found no committed secrets.

Cost

- Local extraction is effectively zero. `COPY INTO` and Snowpark transformations need warehouse compute; Cortex usage and warehouse credits are tracked separately.
- XS compute, auto-suspend, short sessions, optional Tasks kept suspended while idle, and temporary clones dropped.

- Any hours or credits in the plan are a usage scenario, not a bill forecast: region, edition, trial balance, compute minimums, pricing, and activity vary.
- **Measured:** the XS warehouse auto-suspends after 60 seconds; one fully reported warehouse hour used 0.088875 compute credits (shared, not a per-demo bill); the eleven measured Cortex calls used 0.003948756 AI credits; storage was not measured.

Operations

Targets and sample periods are set before measuring. These are project acceptance measures, not production SLOs.

Measure	Target and result
Fetch-to-curated latency, p95	Target under 24 hours. Measured over 179 attempts: 35.9 hours, a miss , mostly because the backfill was curated manually in bulk. The one batch processed as it landed took 293 seconds
Scope of the latency measure	Covers this local pipeline only; excludes USGS publication delay
Evidence	Distribution, sample dates, sample count, and the actual query/run evidence are shown

Reported separately: last successful batch age; newest source-updated age; fetch-to-curated latency; requested, loaded, processed, and rejected rows; pending batches; processing-attempt status; duplicate fact keys; window count and reconciliation results; and coverage gaps.

Project setup

One Python package managed by uv, with SQL applied in numbered order. Setup from a clean clone to a first run took 3 minutes 15 seconds.

Repo layout

```
quakewatch/
├── README.md, CHANGELOG.md, LICENSE, Makefile
├── pyproject.toml, uv.lock, snowflake-config.example.toml
├── src/quakewatch/      extractor, loaders, Snowpark procedure code
├── sql/
│   ├── setup/          01-10, applied in number order
│   └── load/, checks/, analysis/, demos/
├── scripts/
│   ├── pipeline/, checks/, fixtures/, migrations/
│   └── evidence/       one-off Phase 2 and Phase 4 drills
├── tests/              340 offline tests and fixtures
├── docs/              design, data dictionary, results, operations
│   ├── adr/           ten architecture decision records
│   ├── evidence/      dated results log and run records
│   └── plans/         working plans for drills
└── .github/           CI, integration workflow, Dependabot, templates
```

This is the layout at v0.1.0. The `README` and `docs/operations-reference.md` explain each part.

Build plan

Phases were finished in order. Estimates are from the 29 Sep plan; outcomes are as of 2 Oct 2026.

Phase	Main work	Exit evidence	Outcome
0. Source and account check (0.5 evening)	Public city-center coordinates for the three sites; 250 km radius and five-year horizon; verify FDSN parameters, Snowflake role, stage, and Snowpark runtime; save fixtures	Example config and environment note with verified access and actual blockers	Met on 29 Sep: sites, radius, horizon, fixtures, role, stage, Snowpark runtime, and key-pair sign-in verified. Evidence: Phase 0 environment check
1. Batch extract and raw load (1–1.5 evenings)	Origin-time history load, update-time sweep, overlap, count-sized windows, attempt manifest, internal stage, <code>COPY INTO</code> , raw VARIANT table	A repeatable batch appears in RAW with matching attempt and load counts	177 of 180 windows loaded and reconciled (216,361 rows); windows 12, 42, and 73 are source gaps; update sweep built but not completed. Evidence: capture and RAW loads
2. Warehouse models (1.5–2 evenings)	Snowpark validation, rejects, deduplicated revisions, facts, dimensions, current view	Revision, old-event update, tombstone, and rerun fixtures produce expected results	Met: all fixtures passed; all 177 history attempts processed. Evidence: Phase 2 exit review

Phase	Main work	Exit evidence	Outcome
3. Quality and evidence (1–2 evenings)	SQL uniqueness and reconciliation checks, coverage and health views, CI, sample analysis	Actual run evidence and results documented separately from targets	Checks, CI, and sample analysis passed; the 24-hour p95 latency target was missed (35.9 h). Evidence: Phase 3 exit review
4. Optional demos (1–2 evenings)	Clone/Time Travel, Cortex evaluation, documentation polish	Each optional demo has evidence, limitations, and actual usage data	Met: clone and Time Travel drill passed; 9 of 9 evaluated Cortex briefs passed; usage measured. Evidence: Phase 4 close-out

Next: v0.2.0 milestone

- Complete the catalog-wide update sweep and commit the first watermark, or record the unanswerable interval as a documented gap (issue #4).
- Capture or formally close history windows 12, 42, and 73 (issue #5).
- Run extract → load → process → check on a schedule with failure alerts, and measure steady-state latency against a predeclared target (issue #6).
- Run the end-to-end procedure test in a disposable zero-copy clone (issue #7).
- Redeploy the procedure from the ruff-formatted code and record its SHA-256; behaviour is identical, but the deployed bundle differs byte-for-byte (issue #8).
- Validate usefulness with one target analyst using the target-user interview guide in `docs/target-user-interview.md` (issue #9).

Acceptance checklist

The demo script with measured figures is `docs/demo.md` in the repository.

Status on 2 Oct 2026: 12 of 13 complete.

Source and loading

- ☒ Choose public sites and radii and state limitations.
- ☒ Save normal, revision, deletion, malformed, additive-field, over-limit, count-mismatch, and incomplete-window fixtures.

- ☑ Implement per-site origin-time windows plus the catalog-wide `updatedafter` sweep, overlap, query-window audit, and append-only attempt/process manifests. The live sweep has not completed (issue #4).
- ☑ Create the Snowflake stage, `RAW_EVENT_RECORDS VARIANT` table, role, and key-pair auth outside Git.
- ☑ Load JSONL files with `COPY INTO`; reconcile files, query windows, and counts.

Warehouse model

- ☑ Create typed staging, reject handling, dimensions, revision fact, site bridge, batch fact, and current view.
- ☑ Implement alias/tombstone rules, revision ranking, distance, and dedupe-before-MERGE.
- ☑ Demonstrate an idempotent overlapping batch, old-event update capture, transform retry, and current-key uniqueness. Old-event capture is proven on fixtures only.

Quality and evidence

- ☑ Add data-quality, coverage, and health queries.
- ☑ Measure a stated sample; publish actual results with dates and evidence.
- ☑ Run GitHub Actions fixture CI; link only a real status badge.

Product and optional demos

- ☐ Ask one target user if historical comparison is useful. Deferred: no participant recruited yet (issue #9).
- ☑ Complete clone/Time Travel and Cortex only after core batch recovery works.

Known limitations

These are stated in the README rather than hidden.

- The catalog-wide update sweep has not completed, so no watermark has been committed and old-event revisions are proven only on fixtures.
- History windows 12, 42, and 73 are USGS source gaps.
- Latency was measured only on a manually curated backfill; all runs are manual.
- The live integration check compiles SQL with EXPLAIN but does not run the procedure, so MERGE, transaction, and rollback behaviour are proven by recorded drills.
- No target analyst has tested the comparison task, so the product framing is a hypothesis.
- A transactionally consistent USGS catalog snapshot remains unverified; a successful run does not prove complete coverage.

- Storage cost was not measured, and the optional local metrics endpoint was not built.

Close-out

Delivered: the v0.1.0 release with a changelog, reproducible setup (3 minutes 15 seconds from a clean clone to a first run), CI on every push and pull request, measured warehouse and AI usage, completed retry evidence, documented coverage, ten architecture decision records, and a lessons-learned section.

Estimate versus actual: 6–8 evenings were planned; the core build took four calendar days, with hours not tracked. Remaining work is tracked in the v0.2.0 milestone (issues #4–#9).

Scope of the evidence: QuakeWatch is one focused Snowflake batch warehouse: extraction, SQL and data modeling, Python processing, quality checks, backfills, and reproducibility. It has not been run as a scheduled or production service, and its results are limited to the runs recorded in the evidence log.

Sources

Official references used for planning. Recheck service behavior, versions, pricing, and account compatibility before building. Project repository: github.com/kaarthikmohan/quakewatch (release v0.1.0).

- USGS source and attribution: [credit policy](#), [GeoJSON feeds](#), [feed FAQ](#), [ComCat](#), and [FDSN Event API](#)
- Snowflake batch loading and modeling: [COPY INTO table](#), [loading local files](#), [Python procedures](#), [Snowpark setup](#), [package catalog](#), and [Tasks](#)
- Transactions and merge: [transactions](#) and [MERGE](#)
- Recovery: [Time Travel](#), [AT/BEFORE](#), and [object cloning](#)
- Cortex: [AI COMPLETE](#), [regional availability](#), [usage history](#), and [costs](#)
- Alternatives: [NWS API](#) and [OpenAQ API](#)